SOFTWARE REQUIREMENTS SPECIFICATION

for

Real-Time Online Examination System with Proctoring and Suspicious Event Detection

Version 1.0

Prepared by : 1. **Adarsh Bellamane (241IT004)**
2. **Harshith Vellapha (241IT033)**
3. **Rushi Hiren Patel (241IT065)**

Department of INFORMATION TECHNOLOGY
NATIONAL INSTITUTE OF TECHNOLOGY
KARNATAKA, SURATHKAL

August 4, 2026

Contents

List of Tables

List of Figures

1 Introduction

1.1 Purpose

This document specifies the software requirements for the Real-Time Online Examination System with Proctoring and Suspicious Event Detection. It is written for the course instructor/evaluator, the development team, and anyone who maintains the system. The system supports secure online assessments and programming contests through a reliable testing environment, and includes automated proctoring that keeps false positives low, question shuffling based on seating plans, and telemetry tracking of candidate behaviour.

1.2 Scope

The system is a web application with a completely decoupled frontend and backend. It covers the exam lifecycle - registration, Two-Factor Authentication (2FA), device checks, and delivery of randomized questions. While the exam runs, a trained AI model watches the candidate's video feed and flags suspicious behaviour, keeping false positives low. It also records code editor activity and tracks browser-tab focus. The backend is the central authority and uses a real-time cloud database (e.g., Firebase) for results and telemetry.

The scope was decided by looking at how existing online examination platforms are usually built [2], then adapting that to support objective assessments and programming contests together. This includes exam delivery for candidates, configuration and review tools for administrators, and the integrity-monitoring pipeline running during a live session. Not covered here: in-person invigilation, grading of free-text or handwritten answers, and any native desktop or mobile client, since the platform stays browser-only (Section 2.4).

1.3 Definitions, Acronyms and Abbreviations

Table 1.1 lists the terms, acronyms, and abbreviations used in this document, avoiding confusion when the same terms show up again in the functional and non-functional requirements in Chapter 3.

1.4 References

These sources informed the terminology, standards, and conventions used in this document. In-text citations are shown using bracketed numbers, e.g. [1].

[1] IEEE Computer Society, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998, 1998.

[2] ITSourceCode, "Online Examination System – Project Overview and Database Design," [Online]. Available: https://itsourcecode.com/. [Accessed: Aug. 2026].

[3] Mozilla Developer Network, "Web APIs: MediaDevices.getUserMedia(), Document: visibilitychange event, Window: blur event," [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API. [Accessed: Aug. 2026].

Table 1.1: List of Acronyms and Abbreviations

Term	Definition
2FA	Two-Factor Authentication
API	Application Programming Interface
CSV	Comma-Separated Values, a plain-text tabular file format
DB	Database
JWT	JSON Web Token, a compact token format commonly used to represent an authenticated session
ML	Machine Learning
PDF	Portable Document Format
REST	Representational State Transfer, an architectural style for web APIs
SRS	Software Requirements Specification
UI	User Interface
WebSocket	A full-duplex communication protocol used for real-time data streaming over a single TCP connection

[4] Google Inc., *Firebase Realtime Database Documentation*, [Online]. Available: https://firebase.google.com/docs/database. [Accessed: Aug. 2026].

[5] I. Fette and A. Melnikov, "The WebSocket Protocol," Request for Comments (RFC) 6455, Internet Engineering Task Force (IETF), Dec. 2011.

1.5 Overview

The rest of this document describes the system requirements in detail, covering functional and non-functional aspects. Chapter 2 covers product perspective, core functions, user classes, and constraints. Chapter 3 covers functional requirements, interfaces, and design constraints, focusing on what the system should do rather than how it's implemented.

Requirements are grouped by concern rather than by layer, so registration, exam configuration, integrity monitoring, and response handling are treated together even though implementation may span both. Readers wanting a quick overview can refer to Appendix A, which walks through the workflow from exam creation to post-exam review. The revision history is at the end.

2 Overall Description

2.1 Product Perspective

The Real-Time Online Examination System is a stand-alone web application built on a decoupled architecture, splitting frontend and backend work. It integrates with a cloud database for real-time sync. No native install is needed - the platform uses browser capabilities to capture audio-visual feeds and monitor activity.

The system is not part of a larger institutional suite, but its design borrows conventions common across existing online examination platforms [2], mainly around registration and review dashboards. At a high level, one candidate-facing web client and one administrator-facing web client both talk to a shared backend, the only component reading and writing to the cloud database. This centralizes security and validation logic instead of duplicating it across clients.

2.2 Product Functions

The main functions the system provides are listed below, each covered in more detail as a functional requirement in Section 3.1.

- **Registration & Authentication:** Authenticate users with 2FA before exam access.
- **Exam Configuration:** Let administrators upload seating plans that decide exam distribution.
- **Question Shuffling:** Shuffle questions by seating coordinates, or fully randomize without a seating plan.
- **AI Proctoring:** Analyze video feeds with an ML model that judges behavior over time to cut false positives.
- **Code Telemetry Recording:** Record coding sessions (keystrokes, changes) so they can be replayed.
- **Browser Monitoring:** Track focus, window switching, and tab visibility for session.
- **Device Diagnostics:** Check the candidate's hardware works before the test starts.
- **Real-Time Alerts:** Notify administrators immediately when AI proctoring flags a candidate during a live session.
- **Result Export:** Let administrators export collected responses and reports for offline grading and record-keeping.
- **Fullscreen Enforcement:** Keep the candidate's browser in fullscreen for the exam duration and flag attempts to exit.
- **Accommodation Support:** Let administrators grant approved candidates extended time or other exam accommodations.
- **Inactivity Auto-Logout:** End an idle or abandoned session automatically and flag it for admin review.

2.3 User Classes

The system distinguishes between two main classes of users, each with its own interface and permitted actions.

- **Administrators / Faculty:** Creates tests, configures question banks, uploads seating plans, reviews AI proctoring reports, and watches code editor playbacks for academic integrity. Mainly uses the Admin Dashboard (Section 3.2.1).
- **Candidates / Students:** Register for and take exams within the controlled web environment, using the distraction-free Candidate Interface, monitored automatically for the duration.

Both user classes authenticate through the mechanism described in Functional Requirement F.1, though administrator accounts get elevated privileges unavailable through public candidate registration.

2.4 Constraints

The constraints below limit the development team's design and implementation choices.

- **C.1 Platform** – The system must work entirely as a web application, with no desktop downloads or browser extensions.
- **C.2 Hardware** – The candidate must have a working webcam and microphone meeting minimum resolution and sampling requirements.
- **C.3 Operational** – The AI proctoring model must run efficiently, without excessive bandwidth use or crashing the browser.
- **C.4 Data Retention** – Proctoring recordings and telemetry logs must be retained only for a fixed institutional review period before deletion.
- **C.5 Browser Compatibility** – The system must support only modern evergreen browsers that implement the required media and WebSocket APIs.
- **C.6 Fullscreen API Support** – The candidate's browser must support the Fullscreen API to enter and remain in lockdown mode during the exam.
- **C.7 Session Timeout** – Idle sessions must be ended after a fixed, configurable timeout to free the seat for review.

2.5 Assumptions and Dependencies

Below are the assumptions made while designing this system, plus its external dependencies. These are outside the development team's control, and if any prove wrong, this document may need revision.

- Candidates are assumed to have a stable internet connection for the exam, enough for continuous video and telemetry streaming.
- Candidates are assumed to use a reasonably modern browser supporting the media and communication APIs from Section 3.2.

- The system depends on browsers granting access to media devices such as the webcam and microphone.
- The institution is assumed to supply seating plan data in a consistent, machine-readable format ahead of each exam.
- The system depends on the availability of the third-party cloud database provider throughout the exam window.
- Accommodation requests are assumed to be submitted and approved by the institution before the exam begins.

3 Specific Requirements

3.1 Functional Requirements

This section lists the functional requirements the system shall satisfy, each described with its expected input, output, and relative priority (F.1 through F.12).

F.1 Authenticate and Register Users
Description: The system shall let users sign up for exams and require Two-Factor Authentication (2FA) to log in. During registration the candidate enters details like name and roll number. Logging in afterward needs a valid one-time password or authenticator token, and repeated failed attempts trigger a temporary lockout. Administrator accounts are set up separately, with elevated privileges (Section 2.3).
Input: Candidate registration details (name, roll number/email) and a 2FA one-time password or authenticator token.
Output: A confirmed user account and an authenticated session token used for later requests.
Priority: High

F.2 Verify Device Compatibility
Description: Before letting a candidate enter the exam, the system shall run diagnostic checks on the browser, webcam, and microphone. These confirm the browser version is supported, the webcam gives a usable frame at acceptable resolution, the microphone picks up input, and bandwidth is sufficient for streaming. Candidates who fail a check are shown remediation guidance.
Input: Browser hardware API requests (camera, microphone, network probe).
Output: A diagnostic pass/fail status for each component, along with remediation guidance where applicable.
Priority: High

F.3 Process Seating Plans and Shuffle Questions
Description: The system shall let administrators upload a seating plan. When a candidate logs in, the system uses their roll number and seat coordinates to shuffle questions so adjacent students don't get identical sets. The admin can also choose standard randomization without a seating plan, generating distinct sets by seat position or one pooled set shuffled per candidate.
Input: Seating plan mapping (roll number to seat coordinates), the master question bank, and the selected shuffling mode.
Output: A uniquely ordered, or uniquely composed, question set delivered to each candidate's session.
Priority: High

F.4 AI Proctoring and False Positive Reduction
Description: The system shall capture the candidate's video feed and process it through a trained AI model. To cut false positives, it looks at sustained behavioral patterns over time instead of flagging single-frame anomalies. The model checks gaze direction, head pose, and extra faces within rolling windows, building evidence before flagging, keeping the false-positive rate low while still catching suspicious behavior.

Input: Continuous video stream, segmented into rolling time windows for behavioral analysis.
Output: A cheating-probability score, timestamped flagged events, and an aggregated proctoring report for administrator review.
Priority: High

F.5 Record Code Editor Telemetry
Description: For programming tests, the system shall record all actions inside the code editor (similar to competitive programming platforms), collecting telemetry for playback after the exam. Recorded events include keystrokes, text deltas, paste operations, and run/submit actions, each timestamped, producing a serialized session history administrators can scrub through like a contest replay.
Input: Editor keystrokes, text deltas, paste operations, cursor movements, and run/submit actions.
Output: A serialized, timestamped timeline of code evolution suitable for frame-by-frame playback.
Priority: Medium

F.6 Monitor Browser Focus
Description: The application shall track window focus for the whole session, detecting and logging when the candidate switches tabs, minimizes the browser, or opens another window, using standard browser visibility and focus events [3]. Each infraction is timestamped and added to a focus-loss counter on the Admin Dashboard.
Input: Browser window blur, focus, and visibility-change events [3].
Output: Timestamped out-of-focus infraction logs and an aggregate focus-loss counter per candidate.
Priority: Medium

F.7 Collect and Evaluate Responses
Description: The system shall collect all test responses once the candidate finishes or the timer expires, and save them to the database. It also autosaves in-progress responses to the cloud database [4], lowering the risk of data loss if the connection drops mid-exam.
Input: Candidate response data (selected answers, code submissions) and the submission trigger (manual submit or timer expiry).
Output: A persisted response record in the cloud database, along with a submission confirmation timestamp.
Priority: High

F.8 Send Real-Time Proctoring Alerts
Description: The system shall notify administrators in real time through the Admin Dashboard whenever the AI proctoring model flags a candidate above a configured threshold, allowing timely intervention during an active exam.
Input: A proctoring flag event exceeding the configured cheating-probability threshold.
Output: A real-time notification on the Admin Dashboard identifying the candidate, seat, and flagged behavior.
Priority: Medium

F.9 Export Exam Results and Reports
Description: The system shall let administrators export collected responses, proctoring reports, and telemetry summaries in a downloadable format for offline grading and institutional record-keeping.
Input: An export request specifying the exam session and desired report scope.
Output: A downloadable file (e.g., CSV or PDF) containing the requested results and reports.

Priority: Low

F.10 Enforce Fullscreen Mode

Description: The system shall require the candidate to enter fullscreen mode before the exam starts and shall detect and log any exit from fullscreen for the duration of the session, consistent with Constraint C.6.
Input: Fullscreen API enter/exit events from the candidate's browser.
Output: A fullscreen-status flag and timestamped exit-attempt logs added to the proctoring report.
Priority: Medium

F.11 Apply Candidate Accommodations

Description: The system shall let administrators grant approved candidates extended time or other accommodations, which the system applies automatically to that candidate's exam timer and session rules.
Input: An accommodation record specifying the candidate and the adjustment (e.g., extra minutes).
Output: An adjusted exam timer and session configuration for the accommodated candidate.
Priority: Medium

F.12 Auto-Logout Idle Sessions

Description: The system shall monitor candidate activity during the exam and automatically end a session that stays idle beyond the configured timeout (Constraint C.7), freeing the seat and flagging the session for admin review.
Input: Candidate activity signals (keystrokes, mouse movement, focus events) and the configured idle-timeout threshold.
Output: A terminated session, a freed exam seat, and a flagged entry on the Admin Dashboard.
Priority: Medium

3.2 External Interface Requirements

3.2.1 User Interfaces

The system shall provide an Admin Dashboard for hosting tests, uploading seating plans, and reviewing telemetry. This dashboard has summary analytics per session, a seating plan upload widget with validation feedback, and a review player for scrubbing through a candidate's proctoring flags and code playback together. It shall also provide a distraction-free Candidate Interface with the code editor, questions, and a webcam preview - just the exam timer, question navigator, response area, and a small self-view thumbnail, keeping cognitive load low.

3.2.2 Hardware Interfaces

The system shall interface with the candidate's webcam and microphone through standard browser media APIs, such as `getUserMedia` [3]. No proprietary drivers are needed; any webcam/microphone combination the OS exposes to the browser should work if it meets the resolution and sampling constraints from F.2.

3.2.3 Software Interfaces

The backend shall interface with a cloud database (e.g., Firebase) for real-time data sync and storage [4], persisting user profiles, exam configuration, responses, and proctoring telemetry so the Admin Dashboard stays current.

3.2.4 Communications Interfaces

The system shall use HTTP/HTTPS for standard data transmission and secure real-time protocols (such as WebSockets [5]) for streaming code editor telemetry and proctoring logs between frontend and backend, with all traffic encrypted to protect exam data in transit.

3.3 Other Non-Functional Requirements

NF.1 Security
Description: All communications must be encrypted end to end, and exam content must be protected against tampering through mechanisms like signed submissions and role-based access control, so a candidate can never alter another candidate's data.

NF.2 Availability
Description: The system must support concurrent users scaling during scheduled exam windows without degradation, so backend components should scale horizontally as exam sessions grow.

NF.3 Usability
Description: The candidate interface must be intuitive enough to avoid confusion during a high-stakes assessment, keeping first-time candidate onboarding to a minimum before they can start.

NF.4 Data Retention
Description: Proctoring recordings and telemetry logs must be automatically purged once the institutional review period defined in Section 2.4 elapses, so stored candidate data doesn't accumulate indefinitely.

NF.5 Auditability
Description: Administrator actions on candidate data, such as report reviews and exports, must be logged with a timestamp and admin identity to support accountability.

NF.6 Fairness
Description: Accommodation adjustments must apply consistently and only to the candidates they are approved for, so accommodated candidates are neither disadvantaged nor given an unapproved advantage over others.

NF.7 Session Integrity
Description: An idle or abandoned session must not remain open indefinitely; auto-logout must trigger reliably within a bounded time of the configured timeout so an unattended seat cannot be misused.

3.4 Design Constraints

The following design constraints apply to the overall system architecture and aren't expected to change during implementation.

DC.1 Decoupled Architecture
Description: The system must follow a decoupled architecture, with separate frontend and backend communicating only through well-defined APIs.

DC.2 Web-Only Delivery
Description: The application must stay entirely web-based; no native desktop or mobile app shall be produced.

DC.3 Browser Compatibility
Description: The system shall target only modern evergreen browsers supporting the required media and WebSocket APIs, consistent with Constraint C.5.

DC.4 Fullscreen Reliance
Description: Lockdown behavior relies on the browser's native Fullscreen API rather than a custom sandbox, consistent with Constraint C.6.

Appendix A

A.1 General Workflow

The typical workflow starts with the Administrator setting up an exam: uploading questions and the seating plan. The Candidate then registers and passes device checks. During the test, the system shuffles questions based on seating, while AI proctoring, browser monitoring, and editor telemetry run in the background. Afterward, the Admin reviews responses along with behavioral logs and code playbacks. The steps below show which user class handles each one.

1. **Administrator** – Creates the exam, uploads the question bank, and optionally uploads a seating plan.
2. **Candidate** – Registers for the exam.
3. **Candidate** – Passes the device compatibility check (webcam, microphone, browser).
4. **System** – Shuffles or composes the question set based on seating coordinates or full randomization.
5. **System** – Runs AI proctoring, browser-focus monitoring, and code editor telemetry together throughout the exam.
6. **Candidate / System** – Submits responses manually, or the system auto-collects them when the timer runs out.
7. **Administrator** – Reviews the collected responses, proctoring flags, and code editor playbacks to verify academic integrity.

A.2 Workflow Diagram

Figure A.1 presents the same workflow as a flowchart, showing the branch between manual submission and automatic collection before administrator review.

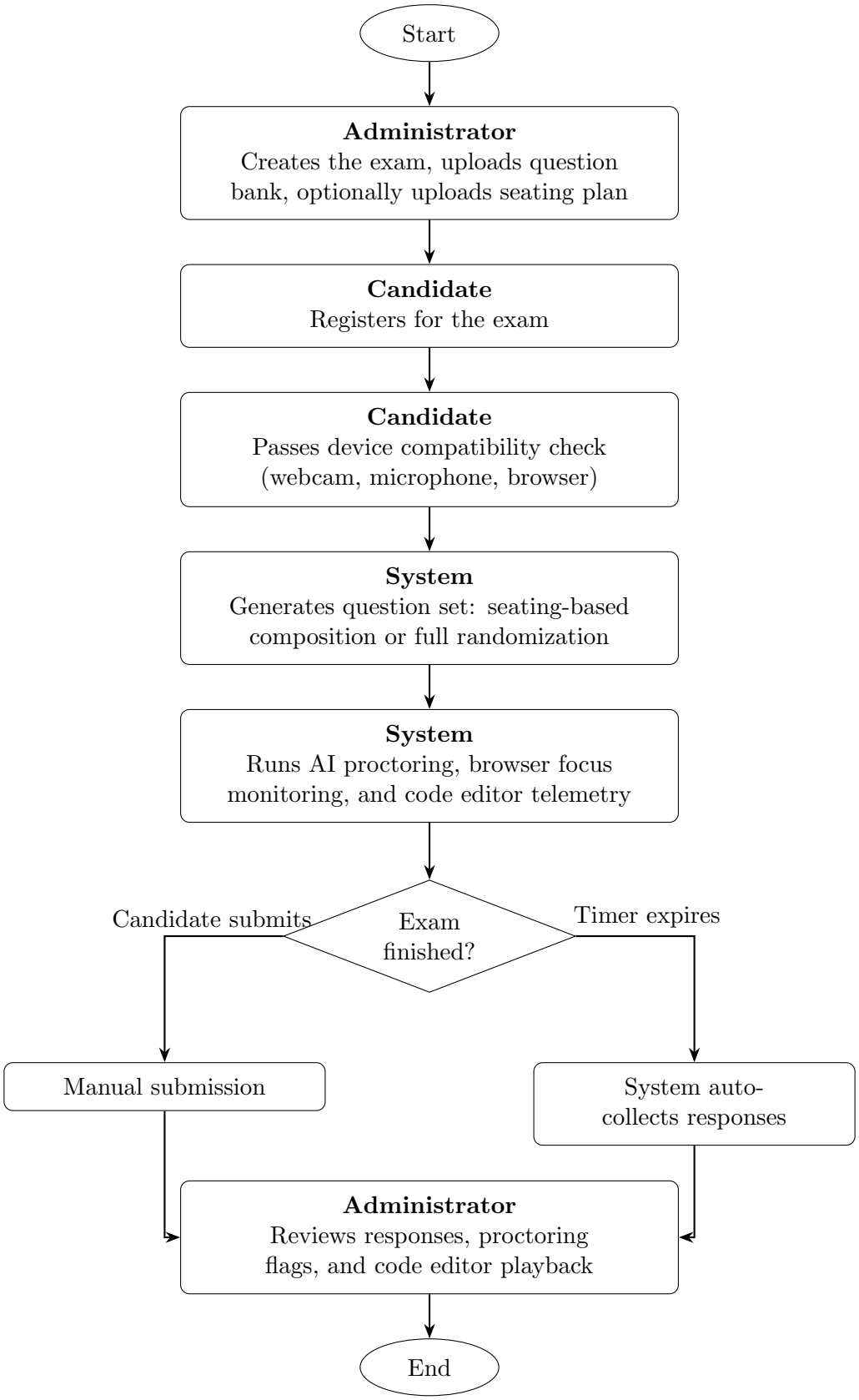

Figure A.1: Exam Workflow Flowchart

Index

Version

Table V.1 keeps track of the revision history of this document.

Table 3.1: Revision History

Version	**Date**	**Description**	**Author(s)**
1.0	–	Initial documentation mapping and structural outlines defined.	Adarsh Bellamane, Harshith Vellapha, Rushi Hiren Patel